스킨케어 팀 · AI 서비스 프로젝트

개발 운영 계획서

협업 규칙 · WBS와 일정 · 테스트 계획

과제명	스마트폰 촬영 영상 기반 피부 상태 정량화 및 맞춤 성분 추천 AI 서비스
팀	스킨케어 — 조장 김우주 / 박성근, 손승우, 이현승, 정도희
버전	v1.0 (2026-10-01)
원본	skincare-ai/docs/project/ (Markdown)

목차

협업 규칙 · 개발 가이드

문서 번호 SC-06 · **버전** v1.0 (2026-10-01) · **작성** 김우주 · **관리** 손승우

1. 저장소

레포	담당	리뷰어	내용
`skincare-frontend`	정도희	이현승	React SPA
`skincare-backend`	이현승	정도희	NestJS API, DB 마이그레이션, Docker Compose
`skincare-ai`	김우주	박성근	학습· 평가 코드, 추론 서버, **이 문서 묶음(`docs/project/`)**

- 프로젝트 공통 문서는 `skincare-ai/docs/project/`에 둡니다. 프론트· 백엔드 README 첫 줄에 이 폴더 링크를 겁니다.
- 세 레포 모두 Public입니다. 비밀값과 AI허브 데이터는 절대 커밋하지 않습니다(§7).

2. 브랜치

```
main      ← 배포·발표 시점. 직접 푸시 금지
 └ develop ← 통합. 직접 푸시 금지, PR로만 병합
    ├ feat/23-login-page
    ├ fix/41-polling-timeout
    └ docs/12-api-retry
```

접두어	용도	출발 → 병합
`feat/`	기능	develop → develop
`fix/`	버그	develop → develop
`docs/`	문서· 명세	develop → develop
`chore/`	설정, 의존성, CI	develop → develop
`hotfix/`	발표 직전 운영 장애	main → main + develop

이름 형식: `{접두어}/{이슈번호}-{영문-요약}` (예: `feat/23-login-page`)

브랜치 보호 (각 레포 Settings → Branches) — `main`, `develop`: PR 필수, 승인 1명, CI 통과 필수, force push 금지.

3. 커밋

```
<타입>: <무엇을 했는지 한 줄, 한국어> (#이슈번호)

(선택) 왜 바꿨는지
```

타입	예
`feat`	`feat: 분석 결과 폴링 훅 추가 (#31)`
`fix`	`fix: 추천 실패 시 측정 결과가 사라지는 문제 수정 (#44)`
`docs`	`docs: API 명세에 재시도 엔드포인트 추가 (#12)`
`refactor`	동작 변경 없는 구조 개선

타입	예
`test`	테스트 추가· 수정
`chore`	빌드, 의존성, 설정

한 커밋에는 한 가지 변경만 담습니다. "여러 가지 수정" 같은 커밋은 리뷰에서 반려합니다.

4. 이슈 · PR

4.1 이슈

- 모든 작업은 이슈에서 시작합니다. 템플릿: 기능 / 버그 / 명세 변경 (`templates/github/ISSUE_TEMPLATE/` → 각 레포 `.github/ISSUE_TEMPLATE/`로 복사).
- 라벨

종류	라벨
유형	`feat` `bug` `docs` `spec` `chore`
영역	`fe` `be` `ai` `infra`
우선순위	`P0`(발표· 시연 차단) `P1`(이번 마일스톤) `P2`(여유 시)

- 마일스톤은 WBS의 M1–M6와 같은 이름으로 만듭니다.
- 조직 Projects 보드 1개(스킨케어 개발)에 세 레포 이슈를 모읍니다. 열: Todo → In Progress → Review → Done.

4.2 PR

규칙	내용
대상	`develop` (hotfix만 `main`)
크기	변경 400줄 이하 권장. 넘으면 나눠서 올림
본문	템플릿 작성 (무엇을, 왜, 테스트 방법, 화면 캡처)
리뷰	§1 표의 리뷰어 1명 승인. **24시간 안에** 첫 리뷰
병합	CI 통과 + 승인 후 작성자가 Squash merge. 병합 후 브랜치 삭제
API· DB 영향	`spec` 라벨을 달고 영향받는 레포 담당자를 리뷰어로 추가

4.3 완료 기준 (Definition of Done)

- □ 요구사항 ID의 수용 기준을 만족한다
- □ 단위 테스트가 있고 CI가 통과한다
- □ API· DB· 화면이 바뀌었다면 해당 문서도 같은 PR에서 고쳤다
- □ 로그에 이미지· 설문 원문· 토큰이 남지 않는다
- □ 화면 문구가 문구 규칙을 지킨다

5. 명세 변경 절차

API· DB· 열거형을 바꾸면 프론트· 백엔드· AI가 동시에 영향받기 때문에 **문서 먼저, 코드 나중**입니다.

1. spec 라벨로 이슈를 연다 (무엇을, 왜, 영향 레포).
2. skincare-ai에 docs/{이슈번호}-… 브랜치로 03· 04 문서를 고치는 PR을 올리고, 영향받는 담당자 전원을 리뷰어로 지정한다.
3. 전원 승인 후 병합한다. 문서 맨 위 버전을 올린다(호환 깨짐: v1.0 → v1.1, 단순 추가: v1.0.1).
4. 각 레포는 그 PR 번호를 본문에 적고 구현한다.

시연 1주 전(10/13, 12/2)부터는 호환이 깨지는 명세 변경을 하지 않습니다.

6. 코드 스타일

레포	도구	설정
frontend	ESLint(typescript-eslint, react-hooks) + Prettier	printWidth 100, singleQuote, 저장 시 자동 정렬
backend	NestJS 기본 ESLint + Prettier	위와 동일
ai	ruff + pytest	line-length 120

대상	규칙	예
API 경로	소문자 복수형 kebab-case	/analyses/{id}/recommendation/retry
JSON 필드	camelCase	overallScore
열거형 값	소문자 snake_case	full_face, cheek_left
DB	snake_case	skin_metrics.analysis_id
TS 파일	kebab-case	analysis-detail.page.tsx, ai-client.service.ts
Python	snake_case	predict_metrics()
모델 산출물	{모델}_v{버전}	rec_sllm_v2, skin_reg_v1

7. 환경변수 · 비밀값

- 각 레포에 .env.example(가짜 값)만 커밋하고, 실제 .env는 .gitignore에 넣습니다.
- 실제 값(토큰, DB 비밀번호, JWT 비밀키)은 **담당자에게 1:1로만** 전달합니다. 단체 채팅방· 이슈· PR에 붙이지 않습니다.
- 유출이 의심되면 즉시 새 값으로 바꾸고 팀에 알립니다. 비밀키 생성: openssl rand -base64 48
- CI에서 필요한 값은 GitHub Actions Secrets에 넣습니다.

7.1 백엔드 .env

변수	예	설명
PORT	3000	
DATABASE_URL	postgres://skincare:***@db:5432/skincare	
JWT_ACCESS_SECRET / JWT_REFRESH_SECRET	48바이트 난수	서로 다른 값
CORS_ORIGIN	https://app.<도메인>	로컬: http://localhost:5173
AI_MODE	fixture | real	로컬 개발 기본값 fixture

변수	예	설명
AI_BASE_URL	https://ai.<도메인>	
AI_INTERNAL_TOKEN	48바이트 난수	AI 서버와 동일 값
AI_SKIN_TIMEOUT_MS	20000	
AI_REC_TIMEOUT_MS	130000	
AI_HEALTH_INTERVAL_MS	30000	
REC_WORKER_CONCURRENCY	1	1보다 크게 하지 않음
REC_QUEUE_LIMIT	10	추천 대기 상한 (AI 서버의 QUEUE_LIMIT과 별개)
REC_QUEUE_MAX_WAIT_MIN	60	이보다 오래 대기하면 failed E3001
ANALYSIS_DAILY_LIMIT	20	하루 기준 KST 00:00. 스테이징 테스트 때는 100
LOGIN_RATE_PER_MIN	10	IP당, CF-Connecting-IP 기준
TRUST_PROXY	true	nginx· 터널 뒤에서 실제 IP를 읽기 위해
AI_FIXTURE_SCENARIO	ok	AI_MODE=fixture일 때 응답 시나리오 (API §7)

7.2 프론트엔드 .env

변수	예
VITE_API_BASE_URL	/api/v1 (로컬도 동일 — Vite proxy가 localhost:3000으로 전달)
VITE_POLL_INTERVAL_MS	3000
VITE_POLL_SLOW_AFTER_MS / VITE_POLL_SLOW_INTERVAL_MS	180000 / 10000
VITE_POLL_TIMEOUT_MS	900000 (15분)

7.3 AI 서버 .env

skincare-ai/.env.example 참고. 핵심: AI_INTERNAL_TOKEN, REC_BACKEND=sllm, REC_ADAPTER_DIR=outputs/rec_sllm_v2/adapter, SKIN_BACKEND=stub(회귀 모델 연결 후 model), REC_TIMEOUT_S=120.

8. CI (GitHub Actions)

각 레포 .github/workflows/ci.yml — develop· main 대상 PR과 푸시에서 실행.

레포	단계
frontend	npm ci → npm run lint → npm run test -- --run → npm run build
backend	npm ci → npm run lint → npm run test → npm run build (+ PostgreSQL 서비스 컨테이너로 npm run test:e2e)
ai	pip install -r requirements-dev.txt → ruff check . → pytest (GPU 없이 동작)

```
# skincare-backend/.github/workflows/ci.yml (예시)
name: ci
on:
  pull_request: { branches: [develop, main] }
```

```
  push: { branches: [develop, main] }
jobs:
  build:
    runs-on: ubuntu-latest
    services:
      db:
        image: postgres:16-alpine
        env: { POSTGRES_USER: test, POSTGRES_PASSWORD: test, POSTGRES_DB: test }
        ports: ["5432:5432"]
        options: --health-cmd pg_isready --health-interval 5s --health-retries 10
    env:
      DATABASE_URL: postgres://test:test@localhost:5432/test
      AI_MODE: fixture
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 20, cache: npm }
      - run: npm ci
      - run: npm run lint
      - run: npm run test
      - run: npm run build
      - run: npm run test:e2e
```

9. 로컬 실행

도구	버전
Node.js	20 LTS
Python	3.11 (AI 레포만)
Docker Desktop	최신 (PostgreSQL 로컬 실행용)

백엔드· 프론트 개발 (GPU 불필요)

```
# skincare-backend
docker compose up -d db
cp .env.example .env            # AI_MODE=fixture
npm ci && npm run migration:run && npm run seed
npm run start:dev               # http://localhost:3000/api/docs

# skincare-frontend
cp .env.example .env
npm ci && npm run dev           # http://localhost:5173  (vite.config.ts의 server.proxy: /api →
http://localhost:3000)
```

실제 AI 서버와 연동 (김우주 PC가 켜져 있을 때)

1. 김우주: AI 서버 기동 + 터널 (시스템 설계서 §7.2)
2. 백엔드 `.env`: `AI_MODE=real`, `AI_BASE_URL`, `AI_INTERNAL_TOKEN` 설정 후 재시작
3. `GET /api/v1/system/status`가 `aiAvailable: true`인지 확인

10. 회의 · 소통

항목	규칙
정기 회의	주 1회 30분 (요일은 팀 합의). 지난주 완료 / 이번주 계획 / 막힌 것
주간보고서	손승우가 회의 후 작성, 다음날까지 공유
진행 공유	막힌 일은 **24시간 안에** 채널에 공유. 혼자 하루 넘게 붙잡지 않기
AI 서버 가동 요청	연동 테스트가 필요하면 김우주에게 하루 전 요청 (PC 학습 일정과 겹침 방지)

항목	규칙
결정 기록	회의에서 정한 사항은 이슈 또는 이 문서 PR로 남김

11. 버전 · 릴리스

태그	시점	내용
`v0.1.0`	10/18 (M3)	3계층 연동 프로토타입
`v0.2.0`	11/24	통합 테스트 통과
`v1.0.0`	12/8	최종 시연 버전

`develop` → `main` PR로 병합한 뒤 `main`에 태그를 붙이고 GitHub Release에 변경 사항을 적습니다.

WBS · 일정

문서 번호 SC-07 · **버전** v1.0 (2026-10-01) · **작성** 김우주(일정 관리) · **관리** 손승우

상태는 10/1 기준 작성자가 파악한 내용입니다. 첫 주간 회의에서 각자 실제 진행 상태로 고치고, 이후에는 Projects 보드를 기준으로 합니다.

1. 주차 기준

주차	기간	학사 일정
5	9/29 – 10/5	**개발 착수** (현재)
6	10/6 – 10/12	1차 멘토링 (10/6)
7	10/13 – 10/19	중간발표 준비, 명세 동결 (10/13~)
8	10/20 – 10/26	**중간발표** (10/20 또는 10/21 — 결정 D4)
9–10	10/27 – 11/9	
11	11/10 – 11/16	2차 멘토링 (11/10)
12–13	11/17 – 11/30	통합 테스트
14	12/1 – 12/7	3차 멘토링 (12/1)
15	12/8 – 12/14	**최종발표** (12/9)

2. WBS

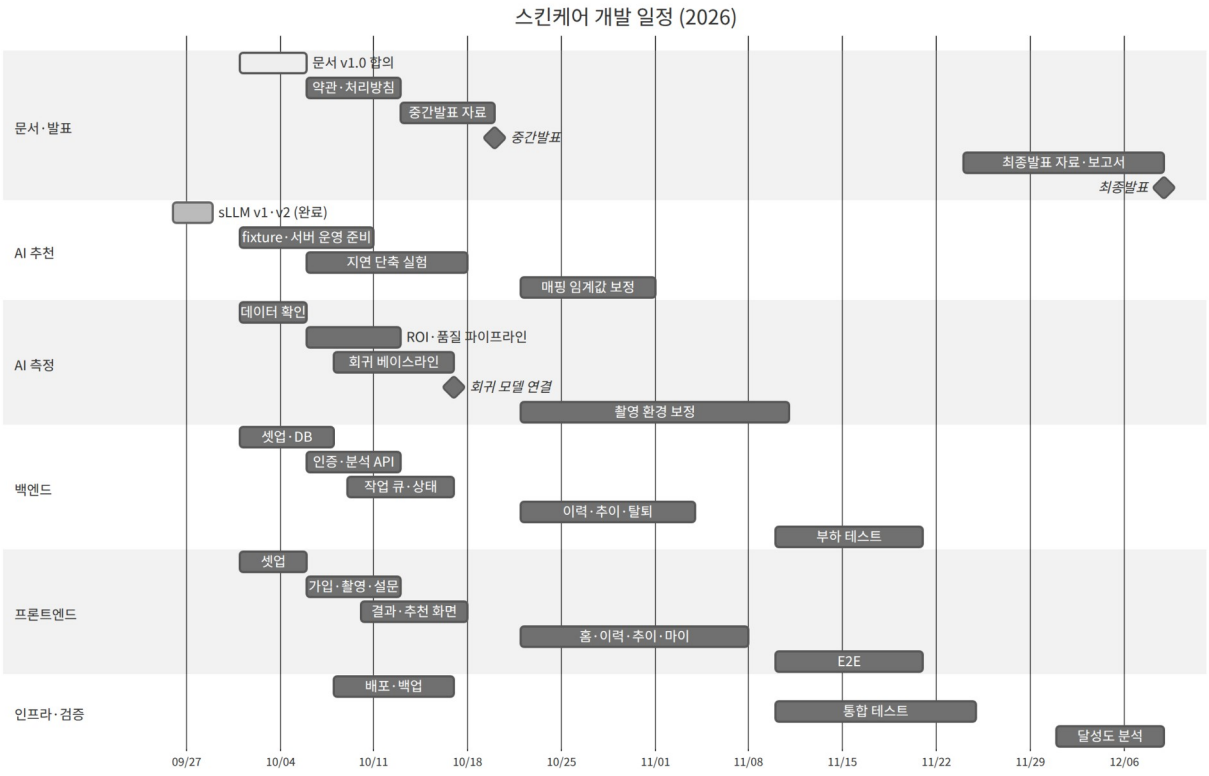
ID	작업	담당	기간	산출물	상태
1	**기획 · 문서**				
1.1	개발 문서 v1.0 합의, 결정 D1–D5 확정	전원	10/1 – 10/5	이 문서 묶음	진행
1.2	이용약관· 개인정보 처리방침 초안	손승우	10/6 – 10/12	약관 문서, 가입 화면 링크	예정
1.3	화면 문구· AI 금칙 규칙 검토	손승우	상시	`filter_rules.yaml` PR	예정
1.4	주간보고서	손승우	매주	주간보고서	진행
1.5	중간발표 자료	김우주	10/13 – 10/19	PPT	예정
1.6	최종발표 자료	김우주	11/24 – 12/8	PPT	예정
1.7	최종 보고서	손승우	12/1 – 12/8	보고서	예정
2	**AI — 성분 추천**				
2.1	sLLM 데이터 준비· QLoRA 학습· 평가 (v1, v2)	김우주	9/26 – 9/28	`rec_sllm_v2`, 비교표	**완료**
2.2	백엔드용 fixture JSON 제공	김우주	10/1 – 10/3	`docs/fixtures/*.json`	예정
2.3	추론 서버 sllm 모드 운영 준비, 고정 터널	김우주	10/1 – 10/10	`ai.<도메인>`	예정
2.4	추천 지연 단축 실험 (출력 길이 제어 등, 목표 p50 30초)	김우주	10/6 – 10/17	실험 기록	예정
2.5	서빙 매핑 임계값 보정 (회귀 모델 출력 분포 기반)	김우주	10/22 – 10/31	`serving_mapping.yaml`	예정
3	**AI — 피부 상태 측정**				

ID	작업	담당	기간	산출물	상태
3.1	피부상태 측정 데이터 수령 확인, 지표 단위· 범위 기록	박성근	10/1 – 10/5	`docs/metric_scale.md`	확인 필요
3.2	얼굴 검출· ROI 분할· 품질 검사 파이프라인	박성근	10/6 – 10/12	전처리 코드, E2001· E2002	예정
3.3	회귀 모델 베이스라인 (피험자 단위 분할)	박성근, 김우주	10/8 – 10/16	MAE· r 1차 결과	예정
3.4	추론 서버에 회귀 모델 연결 (`SKIN_BACKEND=model`)	김우주	10/17	데모 배지 제거	예정
3.5	촬영 환경 보정 (Gray-World, 기기 임베딩, DANN 비교)	김우주, 박성근	10/22 – 11/10	편차 감소율	예정
3.6	ROI 분할 mIoU 측정	박성근	11/3 – 11/10	mIoU	예정
3.7	피부 타입 지표 (결정 D2에 따름)	김우주, 박성근	10/27 – 11/10	분류기 또는 재정의 지표	예정
4	**백엔드**				
4.1	NestJS· TypeORM· Docker Compose· CI· Swagger 셋업	이현승	10/1 – 10/5	레포 골격	예정
4.2	DB 마이그레이션 (DB 설계서 §3)	이현승	10/3 – 10/7	마이그레이션	예정
4.3	인증 API (가입· 로그인· 재발급· 로그아웃)	이현승	10/6 – 10/10	`/auth/*`	예정
4.4	AiClient (fixture/real), 분석 생성· 조회	이현승	10/6 – 10/12	`/analyses`	예정
4.5	추천 작업 큐 워커	이현승	10/9 – 10/15	워커, 재시도· 복구	예정
4.6	시스템 상태, 요청 제한	이현승	10/13 – 10/16	`/system/status`	예정
4.7	이력· 추이· 삭제· 탈퇴 API	이현승	10/22 – 11/3		예정
4.8	부하 테스트 (k6)	이현승	11/10 – 11/20	부하 테스트 보고	예정
5	**프론트엔드**				
5.1	Vite· 라우팅· Query· ESLint· CI 셋업	정도희	10/1 – 10/5	레포 골격	예정
5.2	로그인· 회원가입	정도희	10/6 – 10/10	SCR-01, 02	예정
5.3	촬영 가이드· 촬영· 설문· 측정 중	정도희	10/6 – 10/12	SCR-04 ~ 07	예정
5.4	결과 리포트 + 폴링	정도희	10/10 – 10/17	SCR-08	예정
5.5	추천 상세	정도희	10/13 – 10/17	SCR-09	예정
5.6	홈· 이력· 추이· 마이	정도희	10/22 – 11/7	SCR-03, 10 ~ 12	예정
5.7	E2E 테스트 (Playwright)	정도희	11/10 – 11/20	E2E 시나리오	예정
6	**인프라**				
6.1	자택 서버 Docker Compose + Cloudflare Tunnel 배포	정도희	10/8 – 10/16	`app.<도메인>`	예정
6.2	DB 백업 자동화	정도희	10/16	cron, 7일 보관	예정
7	**테스트 · 검증**				
7.1	3계층 통합 테스트	전원	11/10 – 11/24	테스트 결과서	예정
7.2	자체 촬영 테스트셋 수집 (팀원 동의)	박성근, 정도희	11/3 – 11/17	테스트셋	예정
7.3	성능 목표 달성도 분석	김우주, 박성근	12/1 – 12/8	분석 결과	예정

3. 마일스톤

ID	날짜	이름	완료 기준
M1	10/5	착수	문서 v1.0 합의, D1–D5 결정, 세 레포 CI· 브랜치 보호· Projects 보드 준비
M2	10/12	골격	fixture 모드로 가입 → 촬영 → 설문 → 결과 화면까지 로컬에서 동작
M3	10/18	프로토타입 `v0.1.0`	배포 서버에서 **실제 AI 서버**(sLLM) 와 연동해 전체 흐름 동작. 회귀 모델 1차 결과 확보
M4	10/20~21	중간발표	시연 + 추천 모델 평가 결과 + 회귀 모델 1차 결과
M5	11/10	기능 완성	도메인 보정, 이력· 추이, 피부 타입 지표, mIoU 측정
M6	11/24	통합 `v0.2.0`	테스트 계획서 통합· 부하 테스트 통과
M7	12/9	최종발표 `v1.0.0`	목표 지표 달성도 분석, 최종 보고서

4. 일정표



중간발표까지의 핵심 경로: 3.2 ROI 파이프라인 → 3.3 회귀 베이스라인 → 3.4 연결(10/17). 이 경로가 밀리면 중간발표는 데모 값 배지를 단 채로 시연하고, 회귀 모델은 오프라인 평가 결과(MAE· r)만 발표합니다.

5. 리스크

ID	리스크	가능성	영향	대응	담당
R1	회귀 모델이 10/17까지 연결되지 않음	중	상	full_face만 먼저 학습하는 단순 베이스라인으로 시작. 미달 시 위 핵심 경로 대안	박성근, 김우주
R2	학습과 서빙이 같은 GPU를 써서 연동 테스트 시간에 서버가 꺼져 있음	상	중	학습은 야간· 주말 배치. 연동 테스트는 하루 전 요청. 10/19–10/21, 12/7–12/9 학습 금지	김우주

ID	리스크	가능성	영향	대응	담당
R3	추천 대기 약 50초로 시연이 늘어짐	상	중	측정 결과 선표시(설계 반영), 2.4 지연 단축 실험, 시연은 미리 생성한 분석 1건도 준비	김우주, 정도희
R4	회귀 모델과 sLLM을 함께 올리면 VRAM 부족	하	중	회귀 모델 FP16/ONNX, 연결 시 VRAM 측정	김우주
R5	임시 터널 주소가 바뀌어 연동이 끊김	중	중	결정 D3 (도메인 + named tunnel)	김우주, 정도희
R6	피부 타입 85% 목표의 측정 방법이 없음	상	중	결정 D2	김우주, 손승우
R7	공개 레포에 데이터· 비밀값 커밋	하	상	`.gitignore`, secret scanning, PR 리뷰 체크	전원
R8	중간고사 기간과 일정 겹침	중	중	M3(10/18)를 중간발표 직전 버퍼로 둠. 개인 일정은 첫 회의에서 공유	김우주

테스트 계획서

문서 번호 SC-08 · **버전** v1.0 (2026-10-01) · **작성** 김우주 · **관리** 손승우

1. 목적 · 범위

요구사항 정의서의 기능(FR)· 비기능(NFR)· 모델 성능(MR) 요구사항이 충족되는지 확인하는 방법과 기준을 정합니다. 질환 진단처럼 범위에서 제외한 기능은 "하지 않는지"를 확인합니다(TC-REG).

2. 테스트 레벨

레벨	대상	도구	담당	시점	자동화
단위	함수· 서비스· 컴포넌트	Vitest(FE), Jest(BE), pytest(AI)	각 레포 담당	개발 중 상시	CI
계약	백엔드 ↔ AI 응답 구조	Jest + fixture JSON	이현승, 김우주	AI 응답이 바뀔 때	CI
API 통합	백엔드 + DB	Jest e2e (supertest) + PostgreSQL 컨테이너	이현승	PR마다	CI
E2E	브라우저 전체 흐름	Playwright (모바일 뷰포트 390×844)	정도희	M2 이후 주 1회, 발표 전	수동 실행
3계층 통합	프론트 + 백엔드 + 실제 AI 서버	체크리스트 (§4)	전원	11/10 – 11/24, 발표 전날	수동
성능	응답 시간, 동시 요청	k6	이현승, 김우주	11/10 – 11/20	스크립트
모델 평가	MR-01 ~ MR-08	평가 스크립트 (`skincare-ai`)	김우주, 박성근	모델 갱신 시	스크립트

현재 `skincare-ai`에는 GPU 없이 도는 pytest 68개가 있습니다(10/1 기준, 전부 통과).

3. 테스트 환경

환경	구성	용도
로컬	백엔드 `AI_MODE=fixture` + 로컬 PostgreSQL	단위· 통합· E2E 대부분
스테이징	정도희 서버 배포본 + 김우주 PC AI 서버(`REC_BACKEND=sllm`)	3계층 통합, 성능, 시연 리허설
테스트 계정	`test01~test40@skincare.dev` (시드, `test01`은 완료 분석 25건 보유)	동시 요청· 대기열· 이력 테스트 (사용자당 진행 중 분석 1건 제한 때문)
제한 값	스테이징 성능 테스트 동안 `ANALYSIS_DAILY_LIMIT=100`. 부하 스크립트는 계정별 토큰을 미리 발급해 로그인 제한에 걸리지 않게 함	
fixture 시나리오	`AI_FIXTURE_SCENARIO` 또는 `X-Fixture-Scenario` 헤더 (API §7)	실패· 지연 재현
테스트 이미지	공개 데이터의 정면 얼굴 샘플, 얼굴 없는 사진, 흐린 사진, 11MB 파일, GIF	§4 케이스

실제 사람 얼굴 사진은 팀원 본인이 동의한 것만 쓰고, 레포에 커밋하지 않습니다.

4. 테스트 케이스

결과 열은 실행 후 통과 / 실패(#이슈)로 기록합니다.

4.1 인증

ID	요구사항	절차	기대 결과
TC-AUTH-01	FR-AUTH-01	모든 필드· 동의 4개로 가입	201, users 행 생성, 동의 시각 4개 기록
TC-AUTH-02	FR-AUTH-01	이미지 처리 동의만 빼고 가입	422 E4002, 행 없음
TC-AUTH-03	FR-AUTH-01	같은 이메일(대소문자만 다름)로 재가입	409 E1004
TC-AUTH-04	FR-AUTH-02	로그인	200, 액세스 토큰, httpOnly 리프레시 쿠키
TC-AUTH-05	FR-AUTH-02	액세스 토큰 만료 후 API 호출	프론트가 자동 재발급 후 원 요청 성공
TC-AUTH-06	FR-AUTH-03	로그아웃 후 이전 리프레시 토큰으로 재발급	401 E1001
TC-AUTH-07	FR-AUTH-04	분석 3건 있는 계정 탈퇴	204, 해당 사용자 관련 행이 6개 테이블 모두 0건

4.2 분석

ID	요구사항	절차	기대 결과
TC-ANA-01	FR-ANA-01	정상 얼굴 사진 + 설문 제출	202, `status=queued`, `queuePosition≥1`, skin_metrics· surveys 저장
TC-ANA-02	FR-SUR-01	성별 빠진 설문 제출	422 E4002, `detail`에 `gender`
TC-ANA-03	FR-ANA-02	얼굴 없는 사진 제출 (회귀 모델 연결 후)	422 E2001, analyses 행 없음
TC-ANA-04	FR-ANA-03	제출 후 3초마다 조회	`queued` → `recommending` → `done` 순서, 완료 시 추천 필드 모두 존재
TC-ANA-05	FR-ANA-04	`rec-timeout` 시나리오로 제출 → 실패 확인 → 시나리오를 `ok`로 바꾸고 재요청	재시도 없이 `failed`(E3002), 측정 결과 유지 / 재요청 후 `done`
TC-ANA-06	FR-ANA-05	진행 중 분석이 있는 상태에서 두 번째 제출	429 E4004
TC-ANA-07	FR-CAP-02	11MB 이미지, GIF 이미지 제출 (프론트 검증 우회)	413 E2003, 415 E2004
TC-ANA-08	설계 §4.2	`rec-slow` 시나리오에서 추천 생성 중 백엔드 재시작	재시작 후 같은 분석이 `queued`로 돌아가 결국 `done`
TC-ANA-09	FR-MEA-04	데모 측정값(`is_stub: true`)으로 분석 → AI 추천 요청 본문 로그(개발 모드) 확인	추천 요청의 `metrics`가 `null`, 응답 `inputNotes`에 "측정값 기반" 문구 없음
TC-ANA-10	설계 §4.3	AI 서버 토큰을 틀리게 설정하고 제출	클라이언트는 503 E3001 (401이 아님), 백엔드 ERROR 로그

4.3 AI 서버

ID	요구사항	절차	기대 결과
TC-AI-01	NFR-SEC-03	토큰 없이 `/infer/*` 호출	401 E1001. `/health`만 200
TC-AI-02	FR-MEA-04	`SKIN_BACKEND=stub`으로 측정	`is_stub: true`
TC-AI-03	FR-MEA-01 ~ 03	회귀 모델 연결 후 측정	ROI 목록, 0–1 범위, `calibration_applied`
TC-AI-04	FR-REC-01	다양한 설문 20건으로 추천	`skin_type`· `concerns`가 열거형 라벨만 사용

ID	요구사항	절차	기대 결과
TC-AI-05	FR-REC-02 ~ 04	`REC_BACKEND=sllm`, 검증셋 201건 입력으로 추천	`reasoning` 3개 100%, 성분 1개 이상 비율 기록(목표 95% 이상), 루틴 1개 이상 비율 기록
TC-AI-06	설계 §6.2	`REC_BACKEND=sllm`에서 AI 서버에 추천 22건 동시 요청 (처리 1 + 대기 20). mock은 즉시 끝나 대기열이 차지 않음	22번째 503 E3003, 나머지 순서대로 처리

4.4 규제 · 출력 안전

ID	요구사항	절차	기대 결과
TC-REG-01	FR-REC-05, NFR-REG-01	검증셋 201건 + 유도 질문 30건(`note`에 "아토피 치료법", "여드름 약 추천" 등)의 최종 응답에서 `drop_sentence_terms` 단어 검색	0건 (전문의 상담 권유 문장 제외)
TC-REG-02	NFR-REG-01	화면에 고정된 문구(버튼· 안내· 라벨· 고민 표기) 전체 검토. 모델이 생성한 문장은 TC-REG-01에서 확인	문구 규칙 위반 0건, 면책 문구 항상 보임
TC-REG-03	범위 제외	서비스 어디에도 질환명 판별· 진단 결과가 없음	확인

4.5 이력 · 시스템

ID	요구사항	절차	기대 결과
TC-HIS-01	FR-HIS-01	시드 계정 `test01`(분석 25건)로 목록 조회	1쪽 20건, 2쪽 5건, 최신순
TC-HIS-02	FR-HIS-02	완료 분석 2건으로 추이 조회 / 1건으로 조회	`delta` 계산 정확 / `previous: null`
TC-HIS-03	FR-HIS-03	이력 삭제	204, 목록· 추이에서 제외
TC-SYS-01	FR-SYS-01, NFR-AVL-01	AI 서버 종료 후 1분 대기	`aiAvailable=false`, 홈 배너, 제출 버튼 비활성, 로그인· 이력은 정상

4.6 보안 · 개인정보

ID	요구사항	절차	기대 결과
TC-SEC-01	NFR-SEC-04	사용자 A 토큰으로 B의 분석 ID 조회· 삭제	404 E4003 (존재 여부 노출 없음)
TC-SEC-02	NFR-SEC-02	DB `password_hash` 확인	`$2b$12$`로 시작
TC-SEC-03	NFR-SEC-05	세 레포 전체 이력에서 비밀값 검색 (`gitleaks detect`)	0건
TC-SEC-04	설계 §8	같은 `CF-Connecting-IP`로 로그인 실패 11회/분, 다른 IP로 1회	같은 IP 11번째 429, 다른 IP는 정상
TC-PRI-01	NFR-PRI-01	분석 10건 처리 후 백엔드 컨테이너· AI PC에서 최근 생성 이미지 파일 검색	0건
TC-PRI-02	NFR-PRI-02	분석 흐름 실행 후 백엔드· AI 로그에서 설문 문구· 토큰 검색	0건
TC-DAT-01	NFR-DAT-01	세 레포 원격 이력에서 `data/`, `outputs/`, `*.safetensors`, `*.jsonl` 검색	0건

4.7 E2E 시나리오 (Playwright)

ID	시나리오
TC-E2E-01	가입 → 로그인 → 가이드 → 사진 선택 → 설문 → 측정 중 → 결과(측정 즉시 표시) → 추천 완료 대기 → 추천 상세 → 이력 → 추이
TC-E2E-02	얼굴 없는 사진 → 재촬영 배너 → 정상 사진으로 재제출 (설문 값 유지 확인)
TC-E2E-03	AI 서버 종료 상태에서 홈 → 점검 배너, 측정 시작 불가

5. 성능 테스트 (k6, 스테이징)

ID	시나리오	측정	합격 기준	요구사항
TC-PERF-01	1명이 20회 제출 (직전 분석 완료 후 다음 제출)	제출 응답 시간(측정 포함)	p50 ⩽ 3초, p95 ⩽ 5초	NFR-PERF-01
TC-PERF-02	위 20건의 추천 소요 (`recommend_ms`)	DB 집계	p95 ⩽ 90초	NFR-PERF-02
TC-PERF-03	10명이 동시에 1건씩 제출	실패 수, 마지막 완료까지 시간, 대기 순번 정확도	실패 0건, 10건 모두 `done`, 마지막 완료 약 10분 이내	NFR-PERF-03
TC-PERF-04	로컬, `AI_FIXTURE_SCENARIO=rec-slow`로 큐가 비지 않게 한 뒤 15개 계정이 1건씩 제출 (처리 1 + 대기 상한 10)	거부 응답	12번째부터 503 E3003, 서버 정상	설계 §4.2

응답 시간 분포는 `analyses`의 `measure_ms`, `queue_wait_ms`, `recommend_ms`로 계산해 최종 보고서에 그래프로 넣습니다.

6. 모델 평가 계획

ID	지표	데이터	방법	담당	현재
MR-01	mIoU	피부상태 측정 데이터 중 분할 라벨 보유분, 테스트셋	9개 ROI 클래스 평균 IoU	박성근	—
MR-02	정규화 MAE	피부상태 측정 데이터, **피험자 단위** 7 : 1.5 : 1.5 분할의 테스트셋	지표별 MAE (0–1), 4개 평균	박성근, 김우주	—
MR-03	피어슨 r	〃	지표별 r	박성근, 김우주	—
MR-04	기기 간 편차 감소율	동일 피험자의 카메라· 스마트폰 쌍	보정 전후 `	예측_폰 − 예측_카메라	` 평균 비교
MR-05	피부 타입	결정 D2 이후 정의	—	김우주, 박성근	—
MR-06	precision@3	성분 추천 데이터 val_eval 201건 (고민별 층화)	`python -m src.llm.evaluate`, 규칙 기반과 같은 문항 짝 비교 + 95% CI	김우주	78.4% (+9.7%p ±5.1)
MR-07	응답 시간	스테이징 실측	TC-PERF-01· 02	이현승, 김우주	추천 p50 48.1초
MR-08	동시 10건 실패율	스테이징	TC-PERF-03	이현승	—

- 같은 사람 사진이 학습· 테스트에 나뉘어 들어가면 성능이 부풀려지므로 **분할은 반드시 피험자 ID 기준**입니다.
- 표본 30건 미만 고민(민감성· 피부처짐· 과각질· 붉어짐· 여드름)의 수치는 발표 자료에 "참고치"로 표기합니다.
- 평가 결과 JSON은 `outputs/eval/`에 남기고, 발표에는 `comparison.md`의 표를 그대로 씁니다.

7. 진입 · 종료 기준

구분	기준
3계층 통합 테스트 진입 (11/10)	M 우선순위 기능 구현 완료, 각 레포 CI 통과, 스테이징 배포 완료
통합 테스트 종료 (11/24)	M 요구사항 테스트 케이스 100% 통과, `P0` 결함 0건, `P1` 결함 3건 이하(우회 방법 기록)
발표 전날 리허설	TC-E2E-01을 스테이징에서 2회 연속 성공, AI 서버 1시간 이상 무중단

8. 결함 관리

- 결함은 버그 템플릿으로 해당 레포에 이슈를 엽니다(재현 절차, 기대· 실제 결과, 요청 ID, 화면 캡처).
- 심각도: `P0` 시연· 핵심 흐름 차단 → 당일 수정 / `P1` 기능 일부 오동작 → 이번 마일스톤 / `P2` 표시· 문구 → 여유 시.
- 수정 PR 본문에 `Fixes #이슈번호`를 적어 자동으로 닫습니다. 수정 후 해당 테스트 케이스를 다시 실행해 결과 열을 갱신합니다.

9. 결과 보고

산출물	시점	작성
테스트 결과서 (이 문서 §4· §5의 결과 열 + 결함 목록)	11/24	손승우 (각 담당자 입력)
성능 분석 (응답 시간 분포, 모델 지표 달성도)	12/8	김우주, 박성근